

БГУИР

Кафедра ЭВМ

Отчет по лабораторной работе №1

Тема: «Проектирование комбинационных схем с использованием языка
VHDL»

Выполнил:

студент группы 250504 Казаченко П.Е.

Проверил:

ассистент каф. ЭВМ _____ Шеменков В.В.

Минск

2025

1. Цель работы

Цели работы:

1. Ознакомиться с САПР Xilinx Vivado и приобрести навыки работы с ним.
2. Изучить базовый синтаксис языка VHDL.
3. Приобрести навыки проектирования простейших комбинационных схем на языке VHDL.

2. Исходные данные

В соответствии с полученным вариантом задания необходимо спроектировать комбинационное устройство на языке VHDL. В качестве исходных данных предоставляется схема устройства для реализации. В случае отсутствия табличного описания его работы необходимо выполнить анализ его схемотехники для понимания принципов его работы.

Устройство должно быть реализовано двумя способами:

1. С использованием исключительно логических операторов и параллельного оператора безусловного присваивания.
2. С использованием последовательных операторов за исключением оператора присваивания.

Первый вариант подразумевает запрет на использование других операторов языка VHDL.

Во втором варианте можно использовать любые другие операторы (в том числе и оператор присваивания), но при этом последовательные операторы также должны присутствовать в описании устройства.

После создания описания устройства необходимо разработать для него тестовый модуль, с целью проверки и демонстрации его работы. Тестовый модуль должен выполнять полное тестирование разработанного устройства, что для комбинационного устройства предполагает полный перебор входных значений.

Последним этапом является выполнение поведенческого моделирования устройства с использованием тестового модуля. Моделирование выполняется для всех вариантов описания устройства.

3. Теоретические сведения

Согласно варианту №9 необходимо разработать дешифратор/мультиплексор, соответствующий логической схеме, представленной на рисунке 3.1.

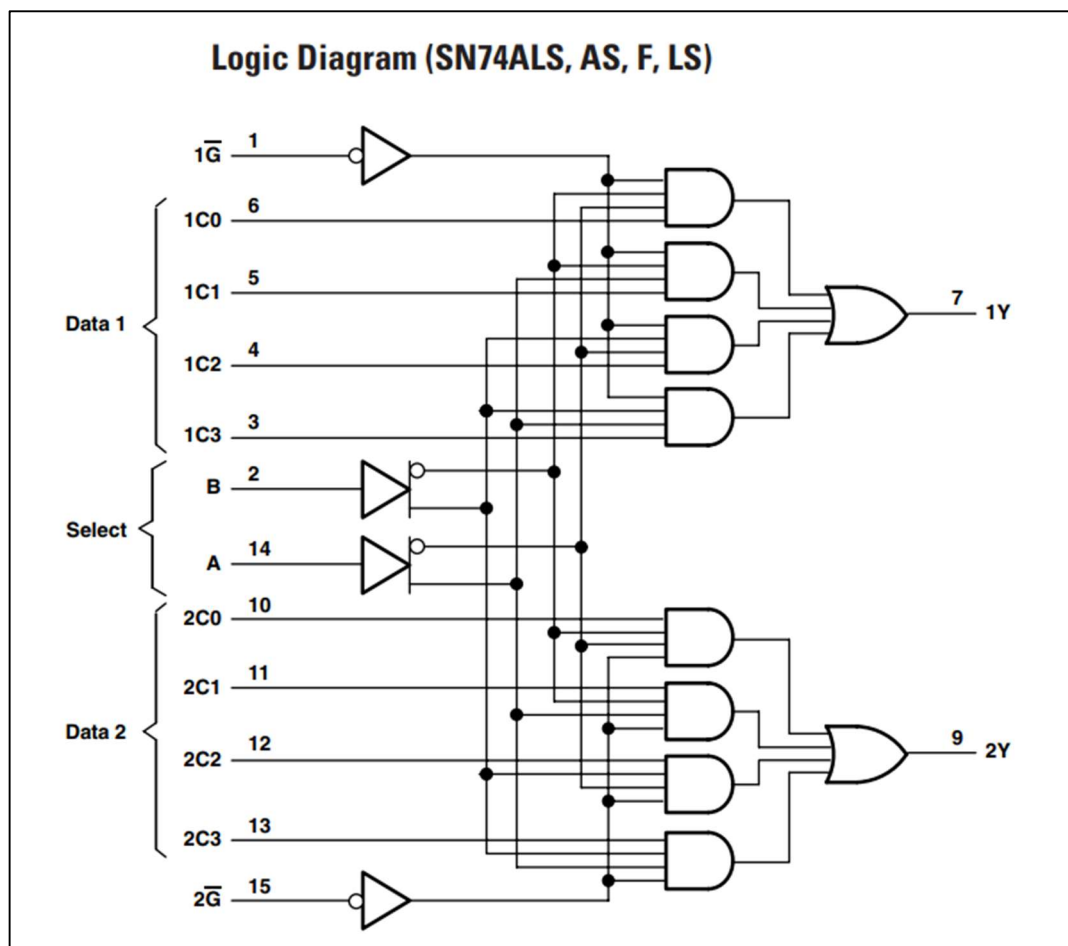


Рис. 3.1 – Заданная по варианту комбинационная схема

Функциональная таблица разрабатываемой комбинационной схемы представлена на рисунке 3.2.

FUNCTION TABLE (SN74)							
SELECT INPUTS		DATA INPUTS				STROBE	OUTPUTS
B	A	C0	C1	C2	C3	$\bar{G}$	Y
X	X	X	X	X	X	H	L
L	L	L	X	X	X	L	L
L	L	H	X	X	X	L	H
L	H	X	L	X	X	L	L
L	H	X	H	X	X	L	H
H	L	X	X	L	X	L	L
H	L	X	X	H	X	L	H
H	H	X	X	X	L	L	L
H	H	X	X	X	H	L	H

Рис. 3.2 – Функциональная таблица разрабатываемой комбинационной схемы

4. Выполнение работы

В соответствии с вариантом №9 задания, используя таблицу истинности и анализ работы комбинационной схемы было описано устройство на языке описания аппаратуры VHDL.

4.1. Листинг кода устройства с использованием последовательных операторов

```
-----
-----
-- Company:
-- Engineer:
--
-- Create Date: 12.09.2025 10:22:51
-- Design Name:
-- Module Name: serial - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-----
-----

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity serial is
    Port (
        not_1g, not_2g : in std_logic;
        a, b : in std_logic;
        c1_0, c1_1, c1_2, c1_3 : in std_logic;
        c2_0, c2_1, c2_2, c2_3 : in std_logic;
        y1, y2 : out std_logic
    );
end serial;

architecture Behavioral of serial is

begin
    process(a, b, not_1g, not_2g, c1_0, c1_1, c1_2, c1_3, c2_0, c2_1, c2_2,
        c2_3)
    begin
```

```

        y1 <= '0';
        y2 <= '0';

        if (not_1g = '0') then
            if (b = '0' and a = '0' and c1_0 = '1' and not_1g = '0' ) then y1
<= '1';
                elsif (b = '0' and a = '1' and c1_1 = '1' and not_1g = '0' ) then
y1 <= '1';
                elsif (b = '1' and a = '0' and c1_2 = '1' and not_1g = '0' ) then
y1 <= '1';
                elsif (b = '1' and a = '1' and c1_3 = '1' and not_1g = '0' ) then
y1 <= '1';
                    end if;
                end if;

            if (not_2g = '0') then
                if (b = '0' and a = '0' and c2_0 = '1' and not_2g = '0' ) then y2
<= '1';
                    elsif (b = '0' and a = '1' and c2_1 = '1' and not_2g = '0' ) then
y2 <= '1';
                    elsif (b = '1' and a = '0' and c2_2 = '1' and not_2g = '0' ) then
y2 <= '1';
                    elsif (b = '1' and a = '1' and c2_3 = '1' and not_2g = '0' ) then
y2 <= '1';
                        end if;
                    end if;

            end process;
        end Behavioral;

```

4.2. Листинг кода устройства с использованием параллельных операторов

```

-----
-----
-- Company:
-- Engineer:
--
-- Create Date: 11.09.2025 09:04:49
-- Design Name:
-- Module Name: parallel - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-----
-----

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

```

```

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity parallel is
  Port (
    not_1g, not_2g : in std_logic;
    a, b : in std_logic;
    c1_0, c1_1, c1_2, c1_3 : in std_logic;
    c2_0, c2_1, c2_2, c2_3 : in std_logic;
    y1, y2 : out std_logic
  );
end parallel;

architecture Behavioral of parallel is
begin

  y1 <= not not_1g and ((not b and not a and c1_0) or
                        (not b and a and c1_1) or
                        (b and not a and c1_2) or
                        (b and a and c1_3));

  y2 <= not not_2g and ((not b and not a and c2_0) or
                        (not b and a and c2_1) or
                        (b and not a and c2_2) or
                        (b and a and c2_3));

  end Behavioral;

```

4.3. Листинг кода тест-борда устройства с использованием последовательных операторов

```

-----
----
-- Company:
-- Engineer:
--
-- Create Date: 12.09.2025 10:38:01
-- Design Name:
-- Module Name: serial_tb - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-----
----

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values

```

```

--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity serial_tb is
-- Port ( );
end serial_tb;

architecture Behavioral of serial_tb is
    -- Объявляем компонент для тестирования
    component serial
        Port (
            not_1g, not_2g : in std_logic;
            a, b : in std_logic;
            c1_0, c1_1, c1_2, c1_3 : in std_logic;
            c2_0, c2_1, c2_2, c2_3 : in std_logic;
            y1, y2 : out std_logic
        );
    end component;

    -- Сигналы для подключения к тестируемому устройству
    signal not_1g, not_2g : std_logic;
    signal a, b : std_logic;
    signal c1_0, c1_1, c1_2, c1_3 : std_logic;
    signal c2_0, c2_1, c2_2, c2_3 : std_logic;
    signal y1, y2 : std_logic;

begin
    -- Создаем экземпляр тестируемого устройства
    uut: serial port map (
        not_1g => not_1g,
        not_2g => not_2g,
        a => a,
        b => b,
        c1_0 => c1_0,
        c1_1 => c1_1,
        c1_2 => c1_2,
        c1_3 => c1_3,
        c2_0 => c2_0,
        c2_1 => c2_1,
        c2_2 => c2_2,
        c2_3 => c2_3,
        y1 => y1,
        y2 => y2
    );

    -- Процесс тестирования
    stimulus: process
    begin
        -- Тест 1: Проверка когда оба выхода отключены
        not_1g <= '1'; not_2g <= '1'; -- Оба выхода отключены
        a <= '0'; b <= '0';
        c1_0 <= '1'; c1_1 <= '0'; c1_2 <= '1'; c1_3 <= '0';
        c2_0 <= '0'; c2_1 <= '1'; c2_2 <= '0'; c2_3 <= '1';
        wait for 10 ns;

        -- Тест 2: Включен только первый мультиплексор
        not_1g <= '0'; not_2g <= '1';
        a <= '0'; b <= '0'; -- Выбираем c1_0
        wait for 10 ns;
    end process;
end

```

```

a <= '1'; b <= '0';  -- Выбираем c1_1
wait for 10 ns;

a <= '0'; b <= '1';  -- Выбираем c1_2
wait for 10 ns;

a <= '1'; b <= '1';  -- Выбираем c1_3
wait for 10 ns;

-- Тест 3: Включен только второй мультиплексор
not_1g <= '1'; not_2g <= '0';
a <= '0'; b <= '0';  -- Выбираем c2_0
wait for 10 ns;

a <= '1'; b <= '0';  -- Выбираем c2_1
wait for 10 ns;

a <= '0'; b <= '1';  -- Выбираем c2_2
wait for 10 ns;

a <= '1'; b <= '1';  -- Выбираем c2_3
wait for 10 ns;

-- Тест 4: Оба мультиплексора включены
not_1g <= '0'; not_2g <= '0';
a <= '0'; b <= '0';  -- Выбираем c1_0 и c2_0
wait for 10 ns;

a <= '1'; b <= '1';  -- Выбираем c1_3 и c2_3
wait for 10 ns;

-- Добавляем проверку разных значений на входах
c1_0 <= '0'; c1_1 <= '1'; c1_2 <= '0'; c1_3 <= '1';
c2_0 <= '1'; c2_1 <= '0'; c2_2 <= '1'; c2_3 <= '0';
a <= '0'; b <= '1';
wait for 10 ns;

-- Завершаем симуляцию
wait;
end process stimulus;

end Behavioral;

```

4.4. Листинг кода тест-борда устройства с использованием параллельных операторов

```

-----
-----
-- Company:
-- Engineer:
--
-- Create Date: 11.09.2025 23:00:55
-- Design Name:
-- Module Name: parallel_tb - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--

```



```
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
```

```
-----

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;
```

```
entity parallel_tb is
-- Port ( );
end parallel_tb;
```

```
architecture Behavioral of parallel_tb is
-- Объявляем компонент для тестирования
component parallel
    Port (
        not_1g, not_2g : in std_logic;
        a, b : in std_logic;
        c1_0, c1_1, c1_2, c1_3 : in std_logic;
        c2_0, c2_1, c2_2, c2_3 : in std_logic;
        y1, y2 : out std_logic
    );
end component;

-- Сигналы для подключения к тестируемому устройству
signal not_1g, not_2g : std_logic;
signal a, b : std_logic;
signal c1_0, c1_1, c1_2, c1_3 : std_logic;
signal c2_0, c2_1, c2_2, c2_3 : std_logic;
signal y1, y2 : std_logic;
```

```
begin
-- Создаем экземпляр тестируемого устройства
uut: parallel port map (
    not_1g => not_1g,
    not_2g => not_2g,
    a => a,
    b => b,
    c1_0 => c1_0,
    c1_1 => c1_1,
    c1_2 => c1_2,
    c1_3 => c1_3,
    c2_0 => c2_0,
    c2_1 => c2_1,
    c2_2 => c2_2,
    c2_3 => c2_3,
    y1 => y1,
    y2 => y2
);

-- Процесс тестирования
```

```

stimulus: process
begin
    -- Тест 1: Проверка когда оба выхода отключены
    not_1g <= '1'; not_2g <= '1'; -- Оба выхода отключены
    a <= '0'; b <= '0';
    c1_0 <= '1'; c1_1 <= '0'; c1_2 <= '1'; c1_3 <= '0';
    c2_0 <= '0'; c2_1 <= '1'; c2_2 <= '0'; c2_3 <= '1';
    wait for 10 ns;

    -- Тест 2: Включен только первый мультиплексор
    not_1g <= '0'; not_2g <= '1';
    a <= '0'; b <= '0'; -- Выбираем c1_0
    wait for 10 ns;

    a <= '1'; b <= '0'; -- Выбираем c1_1
    wait for 10 ns;

    a <= '0'; b <= '1'; -- Выбираем c1_2
    wait for 10 ns;

    a <= '1'; b <= '1'; -- Выбираем c1_3
    wait for 10 ns;

    -- Тест 3: Включен только второй мультиплексор
    not_1g <= '1'; not_2g <= '0';
    a <= '0'; b <= '0'; -- Выбираем c2_0
    wait for 10 ns;

    a <= '1'; b <= '0'; -- Выбираем c2_1
    wait for 10 ns;

    a <= '0'; b <= '1'; -- Выбираем c2_2
    wait for 10 ns;

    a <= '1'; b <= '1'; -- Выбираем c2_3
    wait for 10 ns;

    -- Тест 4: Оба мультиплексора включены
    not_1g <= '0'; not_2g <= '0';
    a <= '0'; b <= '0'; -- Выбираем c1_0 и c2_0
    wait for 10 ns;

    a <= '1'; b <= '1'; -- Выбираем c1_3 и c2_3
    wait for 10 ns;

    -- Добавляем проверку разных значений на входах
    c1_0 <= '0'; c1_1 <= '1'; c1_2 <= '0'; c1_3 <= '1';
    c2_0 <= '1'; c2_1 <= '0'; c2_2 <= '1'; c2_3 <= '0';
    a <= '0'; b <= '1';
    wait for 10 ns;

    -- Завершаем симуляцию
    wait;
end process stimulus;

end Behavioral;

```

5. Выполнение поведенческого моделирования устройства с использованием тестовых модулей

Результат проведения моделирования устройства, описанного с использованием последовательных операторов представлен на рисунке 5.1.

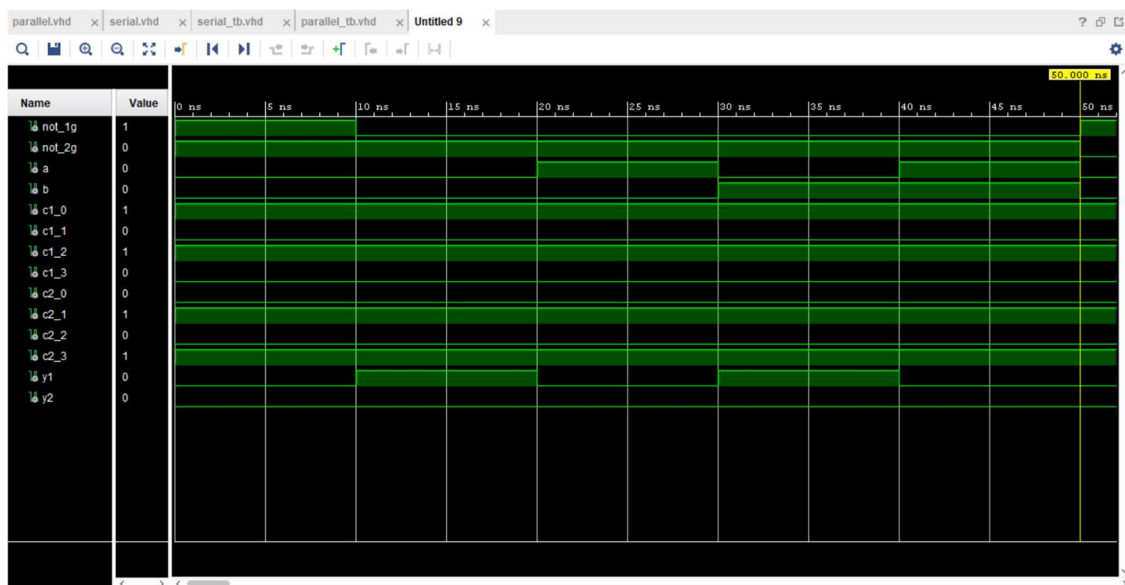


Рис. 5.1 – Результат моделирования устройства, описанного с помощью последовательных операторов

Результат проведения моделирования устройства, описанного с использованием параллельных операторов представлен на рисунке 5.2.

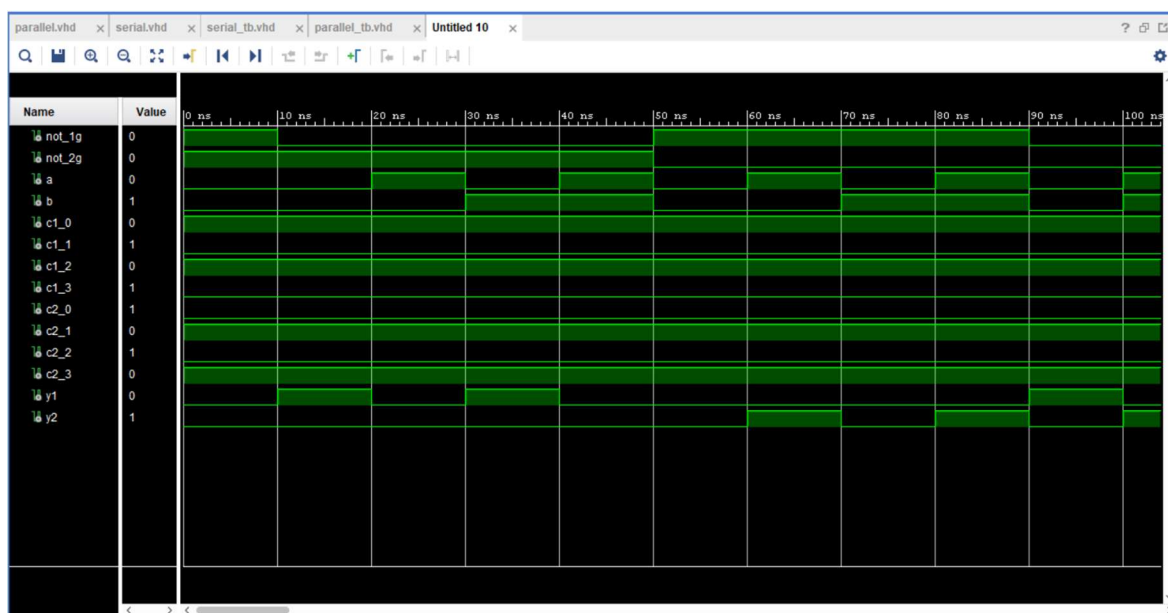


Рис. 5.2 – Результат моделирования устройства, описанного с помощью параллельных операторов

6. Выводы

В процессе выполнения работы было проведено ознакомление с САПР Vivado, изучение базового синтаксиса языка VHDL и заданной комбинационной схемы, а также выполнено ее проектирование на языке VHDL.